



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

**KHOA TOÁN – TIN**

# Lập trình Hướng Đối tượng Object Oriented Programming

\$14 Mẫu thiết kế





# Nội dung

- Giới thiệu
- Các mẫu thiết kế





**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

**KHOA TOÁN – TIN**

# Giới thiệu - mẫu thiết kế

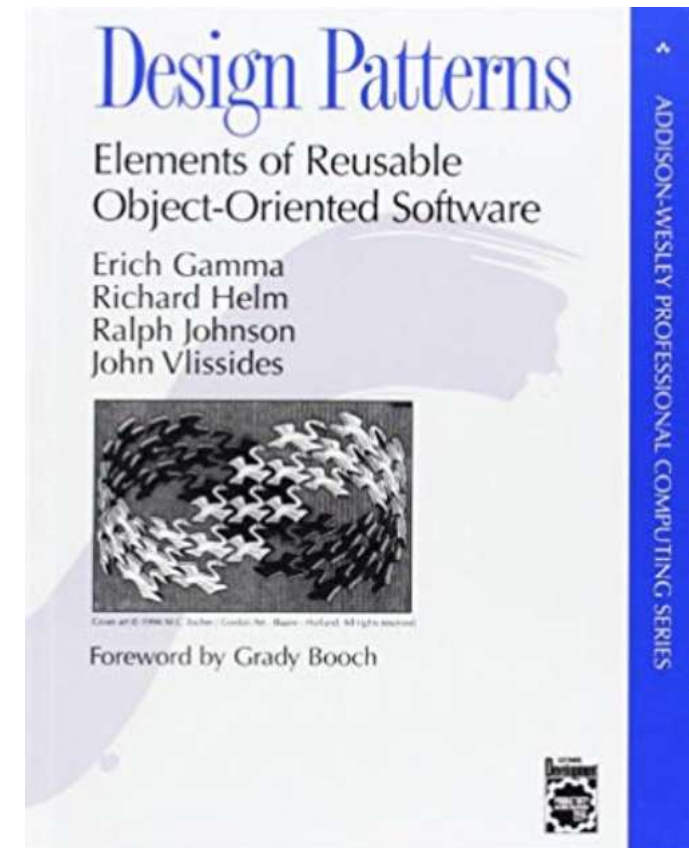


# Mẫu thiết kế

- Mẫu thiết kế (design pattern) là giải pháp thiết kế đưa ra cho các vấn đề thường xuyên gặp phải khi phát triển phần mềm.
- Mỗi mẫu thiết kế thường bao gồm:
  - Tên mẫu
  - Vấn đề cần giải quyết
  - Giải pháp
  - Kết quả

# Ra đời

- Gang of Four: nhóm bộ tứ
  - 4 tác giả Erich Gamma, Richard Helm, Ralph Johnson & John Vlissides xuất bản cuốn "Design Patterns - Elements of Reusable Object-Oriented Software", 1994 => khởi đầu khái niệm mẫu thiết kế
  - Nguyên tắc thiết kế
    - Lập trình cho interface chứ không phải để cài đặt interface đó
    - Ưu tiên tổ hợp (object composition) hơn là kế thừa.



# Nguyên lý

- Lập trình cho interface chứ không phải để cài đặt interface đó
  - Đảm bảo nhất quán
  - Dễ nâng cấp, sửa đổi
- Programming to interface, not on an implementation

```
public interface IMailService
{
    void Send();
}

public class GoogleMailServices : IMailService
{
    public void Send()
    {
        Console.WriteLine("Send by Gmail");
    }
}

public class OutlookMailServices : IMailService
{
    public void Send()
    {
        Console.WriteLine("Send by Outlook");
    }
}

IMailService mailService = new GoogleMailServices();
mailService.Send();
```

# Nguyên lý

- Ưu tiên tổ hợp (object composition) hơn là kế thừa (inheritance)

- Thực hiện mềm dẻo hơn

```
class Car
{
    private IEngine _engine;

    public Car(IEngine engine)
    {
        _engine = engine;
    }

    public void Start()
    {
        _engine.Start();
    }
}
```

```
interface IEngine
{
    void Start();
}

class CombustionEngine : IEngine
{
    void IEngine.Start()
    {
        Console.WriteLine("Starting the combustion engine.");
    }
}

class ElectricEngine : IEngine
{
    void IEngine.Start()
    {
        Console.WriteLine("Starting the electric engine.");
    }
}
```



# Nguyên tắc áp dụng

- Hiểu rõ vấn đề/bài toán
- Hiểu rõ Design Pattern
- Linh hoạt trong việc áp dụng



## Chú ý

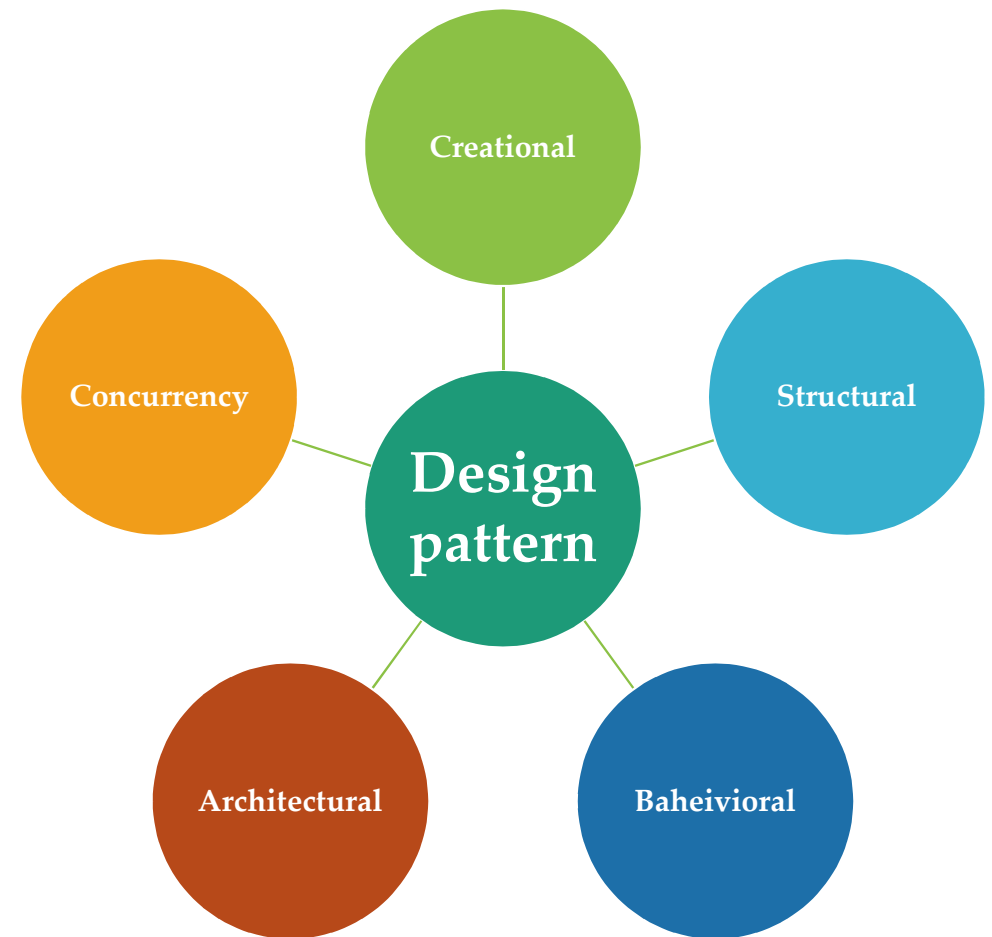
- Không nên áp dụng Design Pattern một cách cứng nhắc
- Không nên lạm dụng Design Pattern
- Cần có thời gian để học hỏi và nghiên cứu Design Pattern

# Lợi ích

- Giải pháp đã được tối ưu hóa, kiểm định => tăng tốc độ phát triển, nâng cao chất lượng, khả năng bảo trì, ...
- Tái sử dụng, khả năng mở rộng
- Nắm bắt code nhanh chóng

# Phân loại mẫu thiết kế

- Có nhiều mẫu, các mẫu cũng liên tục được cải tiến
  - Nhóm mẫu liên quan tới việc tạo đối tượng (Creational Patterns)
  - Nhóm mẫu liên quan tới cấu trúc hệ thống (Structural Patterns)
  - Nhóm mẫu liên quan tới hành vi (Behavioral Patterns)





# Mẫu thiết kế

Gồm

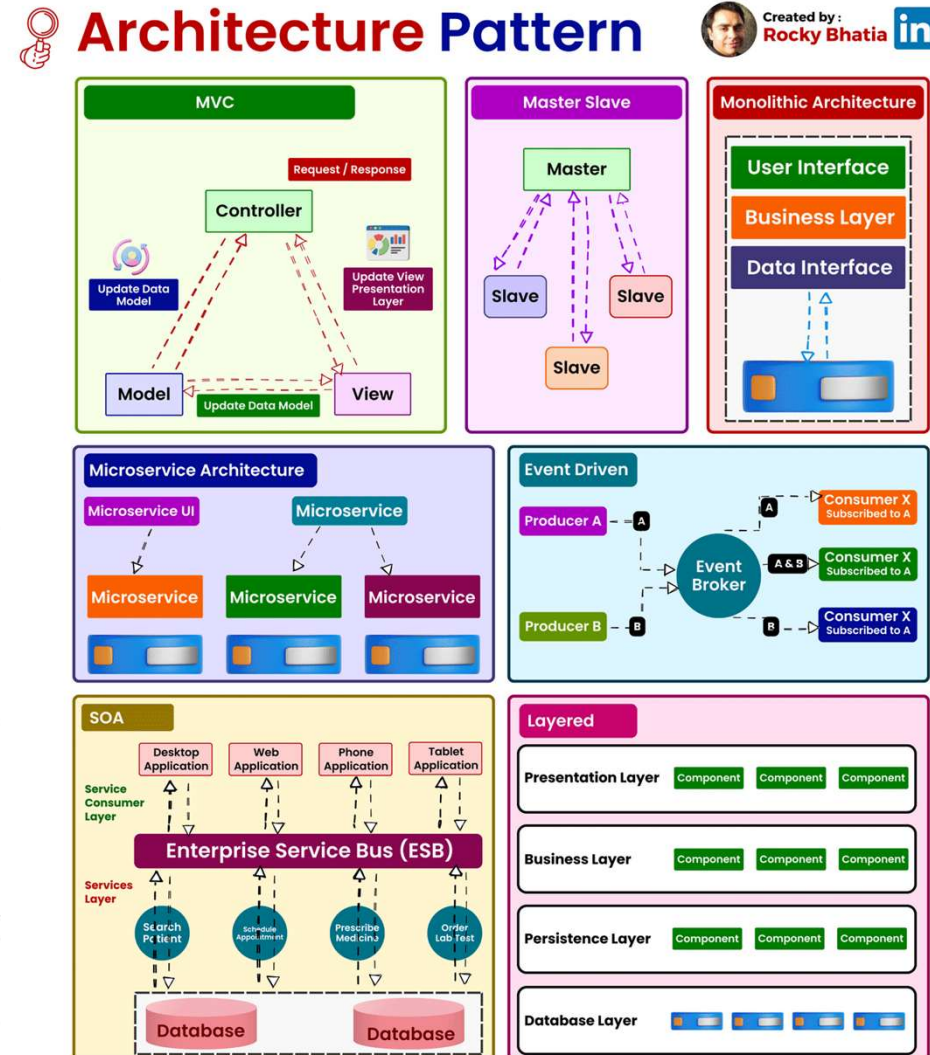
- Tên
- Vấn đề cần giải quyết
- Giải pháp và đặc điểm





# Mẫu kiến trúc

| Software Architecture Pattern |                                                                     |
|-------------------------------|---------------------------------------------------------------------|
| Definition                    | This is a high-level structure of the entire system.                |
| Scope                         | Borad, covers entire system.                                        |
| Purpose                       | Establish entire system layout.                                     |
| Focus                         | System stability, structural organization.                          |
| Documentaion                  | It involves architectural diagrams and high-level design documents. |



| Features            | Software Architecture Pattern                                       | Design Pattern                                                                          |
|---------------------|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <b>Definition</b>   | This is a high-level structure of the entire system.                | This is a low-level solutions for common software design problems within components.    |
| <b>Scope</b>        | Borad, covers entire system.                                        | Narrow, focuses on individual components.                                               |
| <b>Purpose</b>      | Establish entire system layout.                                     | Provide reusable solutions for the recurring problems within a systems' implementation. |
| <b>Focus</b>        | System stability, structural organization.                          | Behavioral and structural aspects within components.                                    |
| <b>Documentaion</b> | It involves architectural diagrams and high-level design documents. | It includes UML diagrams, detailed design specifications.                               |
| <b>Examples</b>     | Layered Architecture, Microservices, Client-Server.                 | Singleton, Factory, Strategy, Observer.                                                 |



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

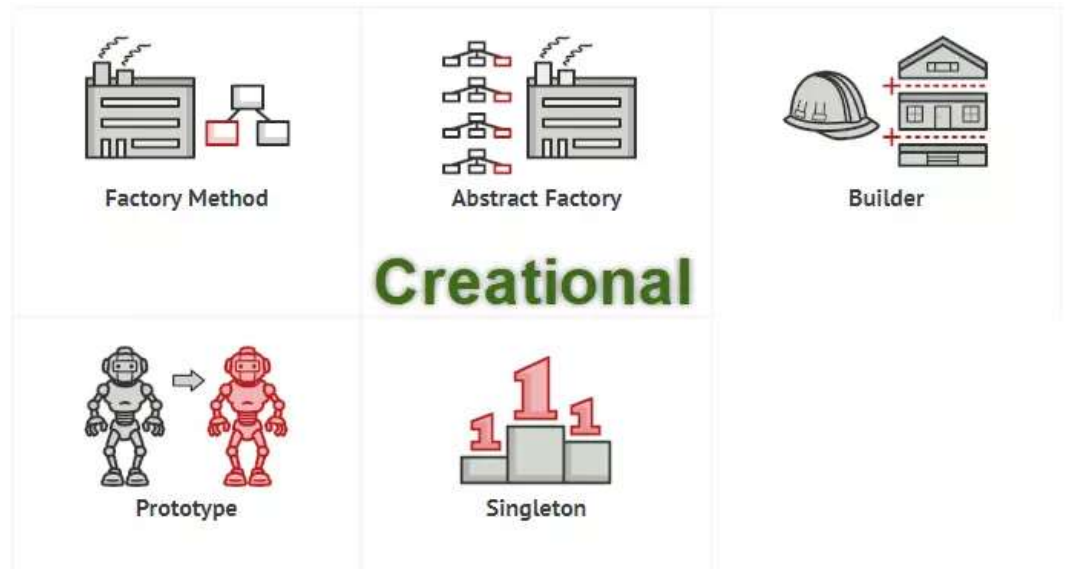
**KHOA TOÁN – TIN**

# Mẫu khởi tạo Creational patterns



# Mẫu khởi tạo

- Creational Patterns cung cấp các giải pháp khởi tạo đối tượng một cách linh hoạt và phù hợp với bối cảnh sử dụng
- Che dấu logic tạo đối tượng



# Factory (method) pattern

## ■ Bài toán:

- Website TMĐT: Thanh toán

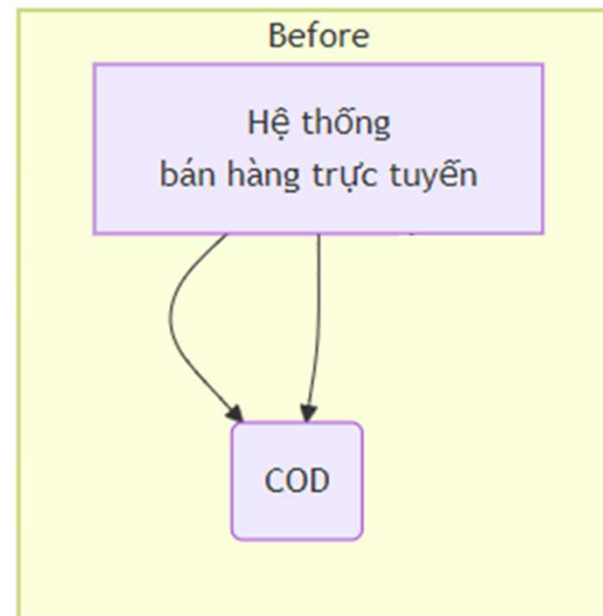
- Ban đầu: COD

=> Viết mã (client) xử lý tt COD

- Phát sinh: tt thẻ

=> Sửa mã - thêm tt thẻ

Phát sinh các hình thức tt khác ?



## ■ Phương án

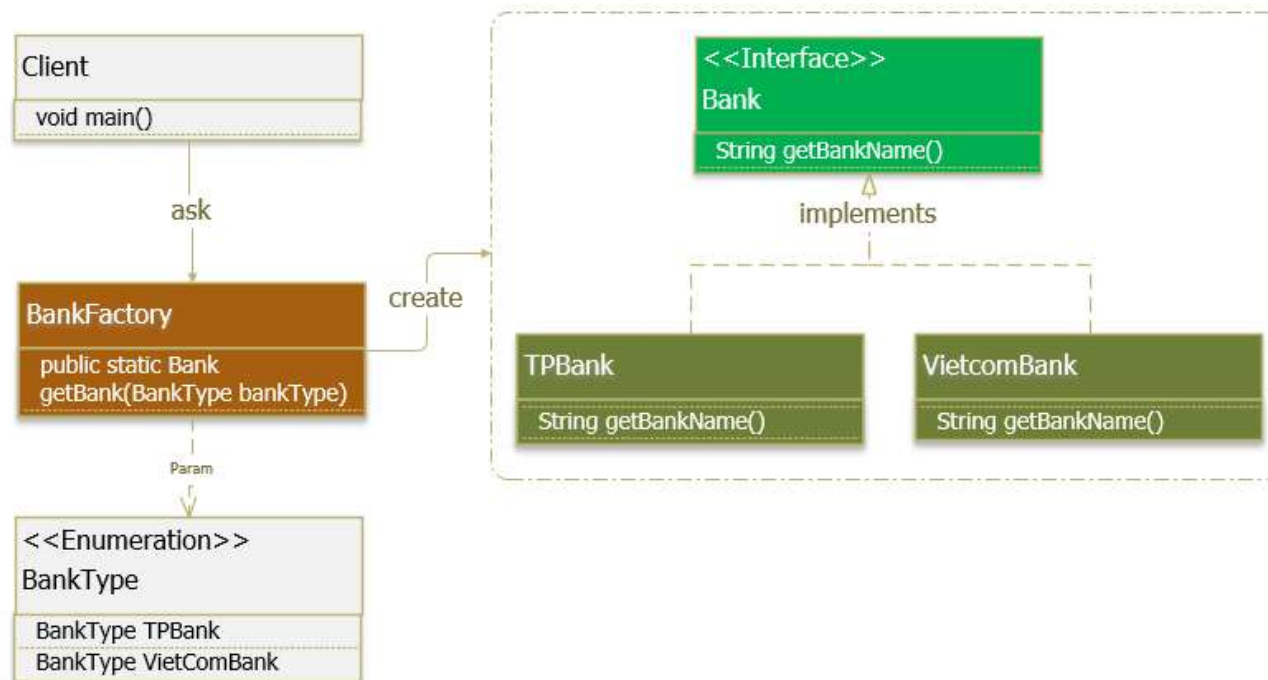
- Factory pattern

# Factory (method) pattern

- Tên
  - Factory Pattern
- Vấn đề cần giải quyết:
  - Tạo đối tượng của lớp phụ thuộc vào dữ liệu đầu vào (ảnh hưởng cách tạo)
- Giải pháp:
  - Tạo lớp Factory, dùng factory method để tạo đối tượng

# Factory (method) pattern

- Cơ bản gồm
  - Super class (Interface/abstract class/class)
  - Sub class: cài đặt các method super class
  - Factory class với factory method-thực hiện tạo



# Factory (method) pattern

## cài đặt vd

```
public interface PaymentMethod
{
    void PerformPayment(decimal amount);
}
// Concrete Products: cash on delivery payment and online payment
public class CODPayment : PaymentMethod
{
    public void PerformPayment(decimal amount) {
        Console.WriteLine($"Processing COD payment of {amount:C} ... DONE");
    }
}
public class OnlinePayment : PaymentMethod
{
    public void PerformPayment(decimal amount) {
        Console.WriteLine($"Processing online payment of {amount:C} ... DONE");
    }
}
```

# Factory (method) pattern

## cài đặt vd

```
public enum PaymentMethodType {  
    COD, Online  
}  
public static class PaymentFactory {  
    public static PaymentMethod GetPaymentMethod(PaymentMethodType paymentMethod)  
    {  
        switch (paymentMethod)  
        {  
            case PaymentMethodType.COD:  
                return new CODPayment();  
            case PaymentMethodType.Online:  
                return new OnlinePayment();  
            default:  
                throw new Exception("Invalid payment method");  
        }  
    }  
}
```

# Factory (method) pattern

## cài đặt vd

```
Order order = new Order();  
order.AddProduct(new Product("Laptop", 1000));  
order.AddProduct(new Product("Mouse", 20));  
order.AddProduct(new Product("Keyboard", 50));
```

```
// Select the payment method: factory method
```

```
PaymentMethod selectedPaymentMethod =  
    PaymentFactory.GetPaymentMethod(PaymentMethodType.Online);
```

```
order.SetPaymentMethod(selectedPaymentMethod);  
order.ProcessOrder();
```

```
PS D:\Projects\source\repos4-hust\oop03> cd "d:\Pro  
D:\Projects\source\repos4-hust\oop03\factory01.cs(6:  
ullable. [D:\Projects\source\repos4-hust\oop03\oop0  
Processing online payment of $1,070.00 ... DONE  
PS D:\Projects\source\repos4-hust\oop03> █
```

## Factory (method) pattern

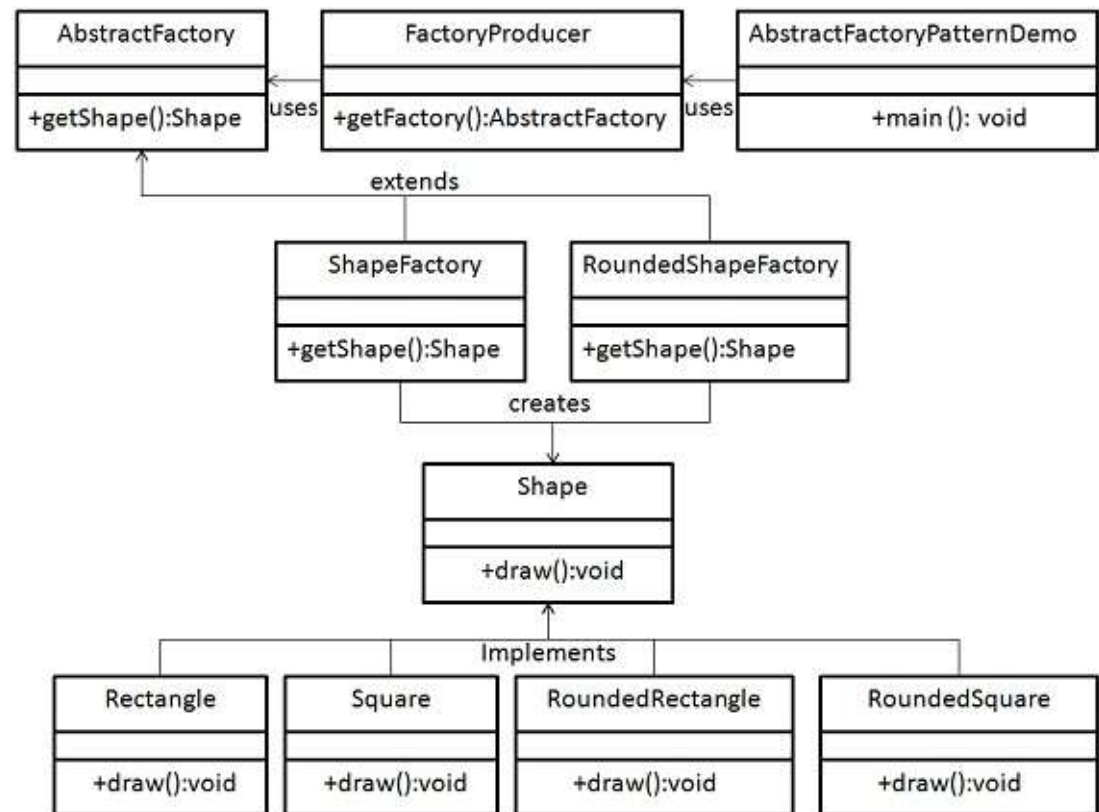
- **Sử dụng Factory Pattern khi nào?**
  - Có nhiều sub class và dựa trên đầu vào, cần trả về một sub class

# Factory (method) pattern

- Lợi ích
  - Factory Pattern giúp giảm sự phụ thuộc giữa các module (loose coupling)
  - Mở rộng code dễ dàng hơn
  - Che dấu logic thực sự được khởi tạo bên dưới phương thức factory

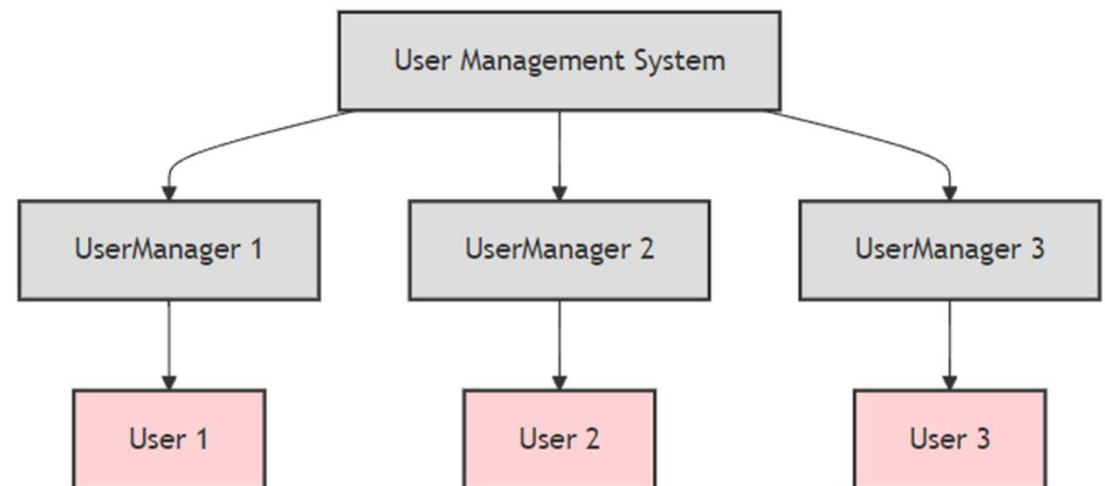
# Factory pattern

## Factory method và Abstract Factory



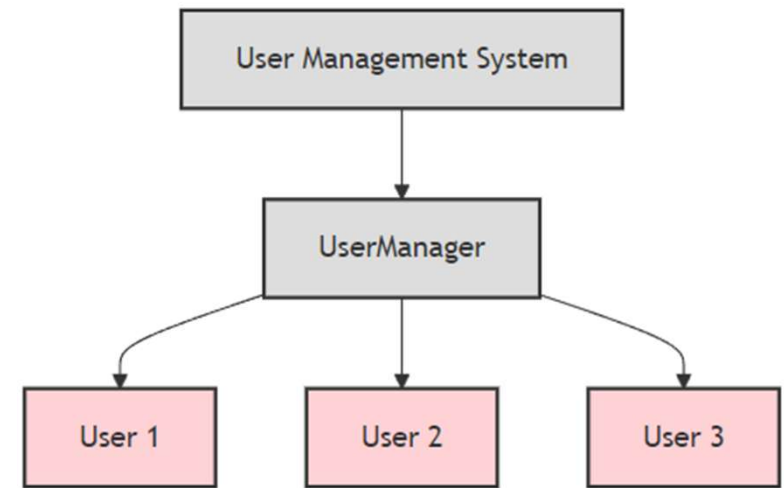
# Singleton pattern

- Nhu cầu chỉ cần có duy nhất 1 thể hiện của lớp
- VD: Quản lý người dùng
  - Nhiều đối tượng quản trị => các vấn đề
    - Dữ liệu bị trùng lặp
    - Xung đột tài nguyên
    - Khó kiểm soát



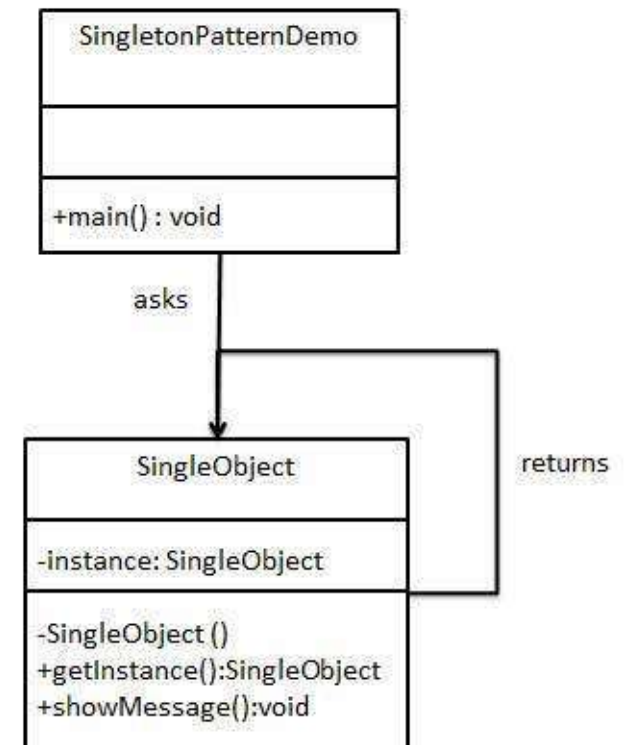
# Singleton pattern

- Giải pháp
  - Chỉ duy nhất 1 đối tượng user manager



# Singleton pattern

- Tên
  - Singleton Pattern
- Vấn đề cần giải quyết:
  - Tạo 01 và chỉ 01 thể hiện của một lớp cụ thể
- Giải pháp:
  - Tạo lớp Singleton



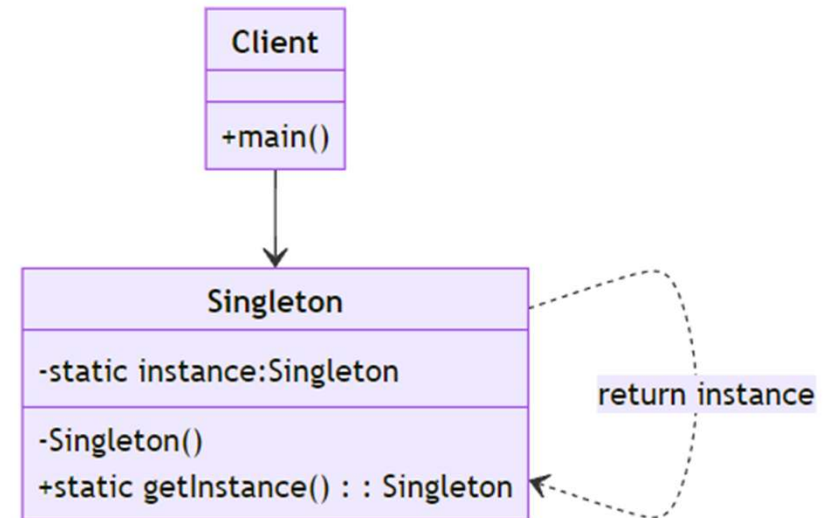
# Singleton pattern

## ■ Giải pháp

### • Lớp singleton

- Phương thức khởi tạo private - hạn chế truy cập từ bên ngoài
- Thành viên static member - tham chiếu đối tượng duy nhất của lớp
- Phương thức static public : trả về tham chiếu đối tượng duy nhất của lớp

=> Đối tượng toàn cục, truy cập được từ bất kỳ đâu trong chương trình



# Singleton pattern

- Triển khai cài đặt
  - VD: eager singleton

```
public class MySingleton
{
    //private constructor
    private MySingleton() { }
    //private static instance
    static readonly MySingleton _instance = new MySingleton();
    //public static property to access the instance
    public static MySingleton Instance => _instance;
    //sample count variable maintain global value
    private int _count = 0;
    public void Increase() => _count++;
    public int Count => _count;
}
```

- + Phương thức khởi tạo private
- + Thành viên static member - tham chiếu đối tượng duy nhất của lớp
- + Phương thức static public : trả về tham chiếu đối tượng duy nhất của lớp

# Singleton pattern

## ■ Triển khai cài đặt

```
MySingleton singleton = MySingleton.Instance;  
Console.WriteLine($"Start: {singleton.Count}");  
singleton.Increase();  
Console.WriteLine($"First increment: {singleton.Count}");  
singleton.Increase();  
Console.WriteLine($"Second increment: {singleton.Count}");  
singleton.Increase();  
Console.WriteLine($"Third increment : {singleton.Count}");
```

```
D:\Projects\source\repos4-hust\oop03\Program.cs  
Start: 0  
First increment: 1  
Second increment: 2  
Third increment : 3
```

# Singleton pattern

## ■ Ứng dụng

Khi cần đúng 01 thể hiện để phối hợp các tác vụ, vd

- Quản lý cấu hình
- Ghi nhật ký Logging
- Caching
- Kết nối CSDL
- ...

# Singleton pattern

## Triển khai

- Eager initialization:

- Đối tượng singleton được khởi tạo sớm - trước khi sử dụng đến

```
public class EagerSingleton {  
  
    private static EagerSingleton _instance = new EagerSingleton();  
    // Private constructor to avoid client applications to use constructor  
    private EagerSingleton() {  
    }  
    public static EagerSingleton getInstance() {  
        return _instance;  
    }  
}
```

# Singleton pattern

## Triển khai

- Lazy initialization:
  - Đối tượng singleton được khởi tạo khi sử dụng đến

```
public class LazySingleton {  
  
    private static LazySingleton instance;  
  
    private LazySingleton() {  
    }  
    public static LazySingleton getInstance() {  
        //Check if NULL  
        if (instance == null) {  
            instance = new LazySingleton();  
        }  
        return instance;  
    }  
}
```

# Singleton pattern

## Triển khai

- Lazy initialization:

- Dùng .Net Lazy<T>

```
public sealed class LazyTSingleton
{
    //This will make the Singleton Instance Lazy Loading
    private static readonly Lazy<LazyTSingleton> SingleInstance = new
    Lazy<LazyTSingleton>(() => new LazyTSingleton());
    public static LazyTSingleton GetInstance()
    {
        //don't need to use any NULL Checking code
        //Value property of the Lazy object => Singleton Instance
        return SingleInstance.Value;
    }
    private LazyTSingleton() {
    }
}
```

# Singleton pattern

## Triển khai

### ■ Thread safe

- Vấn đề: có thể lỗi khi chạy multi thread - race condition => cần thread safe

```
public class LazySingleton
{
    private static LazySingleton instance;
    private LazySingleton() {}

    public static LazySingleton getInstance()
    {
        if (instance == null) //check if NULL => create new instance
        {
            instance = new LazySingleton();
        }
        return instance;
    }
}
```

# Singleton pattern

## Triển khai

- Thread safe

- Giải pháp: có thể dùng lock, Lazy<T>

```
public class Singleton{
    private static Singleton _instance;
    private static readonly object _lock = new object(); // Lock synchronization object
    private Singleton() {}
    // Public static method to get the instance of the class
    public static Singleton Instance{
        get {
            // Double-checked locking to ensure thread safety
            if (_instance == null) {
                lock (_lock) {
                    if (_instance == null) { _instance = new Singleton(); }}}
            return _instance;
        }
    }
}
```

# Bài tập

- Chương trình vẽ
  - người dùng chọn các hình: chữ nhật, vuông, tròn => vẽ
  - dùng Factory pattern để thực hiện



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

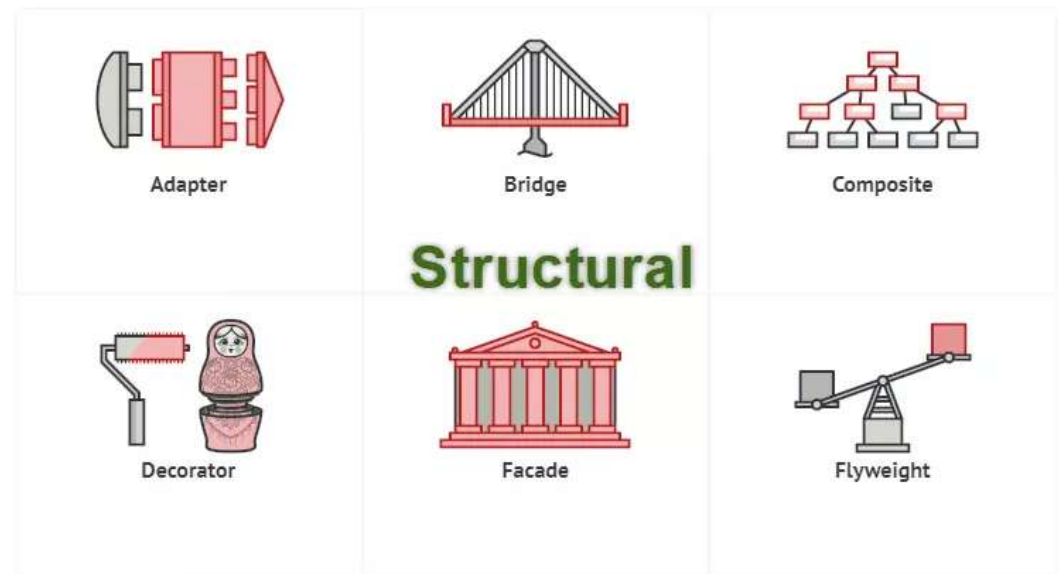
**KHOA TOÁN – TIN**

# Mẫu cấu trúc Structural patterns



# Mẫu cấu trúc

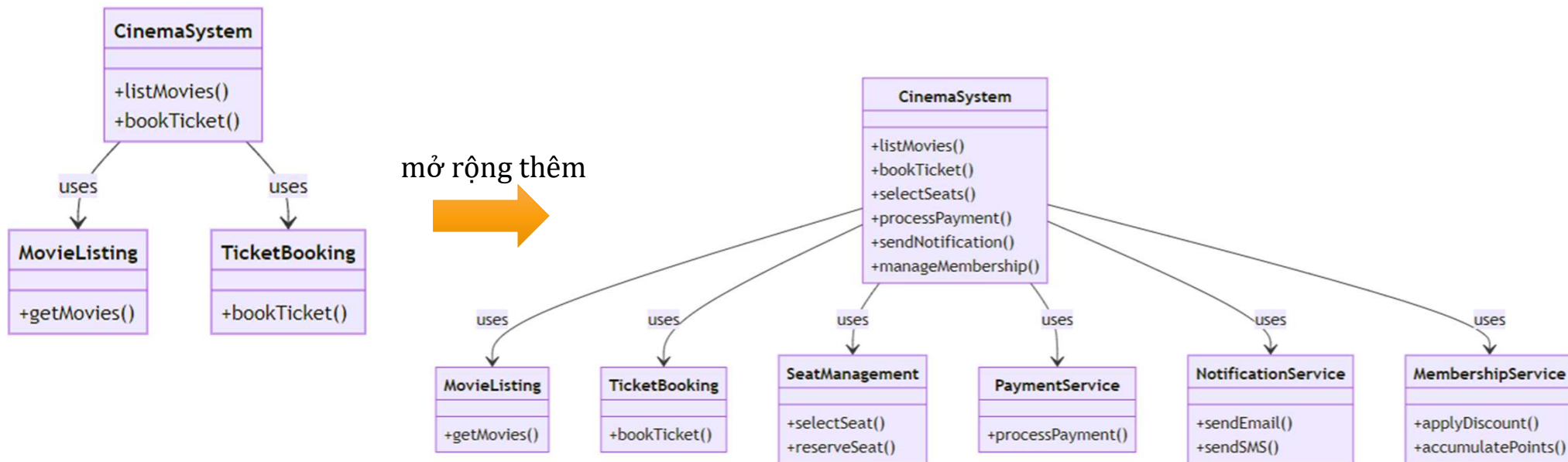
- Structural Patterns liên quan đến cấu trúc và mối quan hệ giữa các lớp và đối tượng nhằm tạo ra cấu trúc phần mềm dễ thay đổi và bảo trì hơn.



# Facade

## ■ VD:

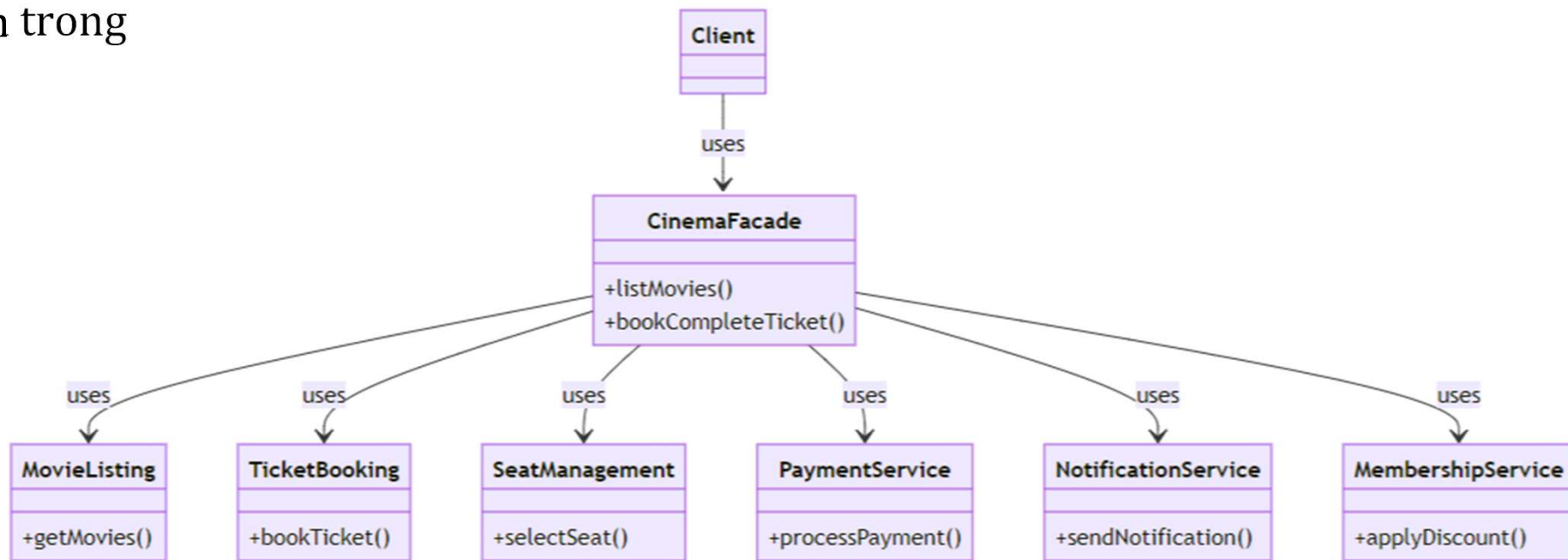
- Hệ thống gồm nhiều hệ thống con phức tạp => client phải biết để khai thác => phụ thuộc chặt, code khó bảo trì



# Facade

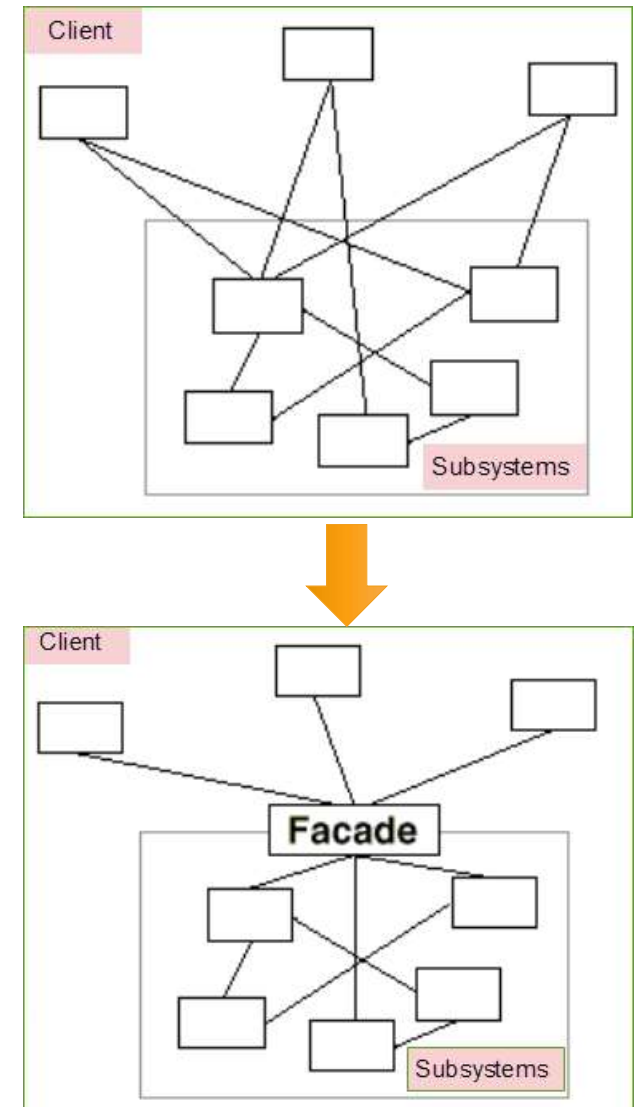
## ■ VD:

- Giải pháp: facade pattern, tập giao diện trung gian đơn giản, che dấu phức tạp bên trong



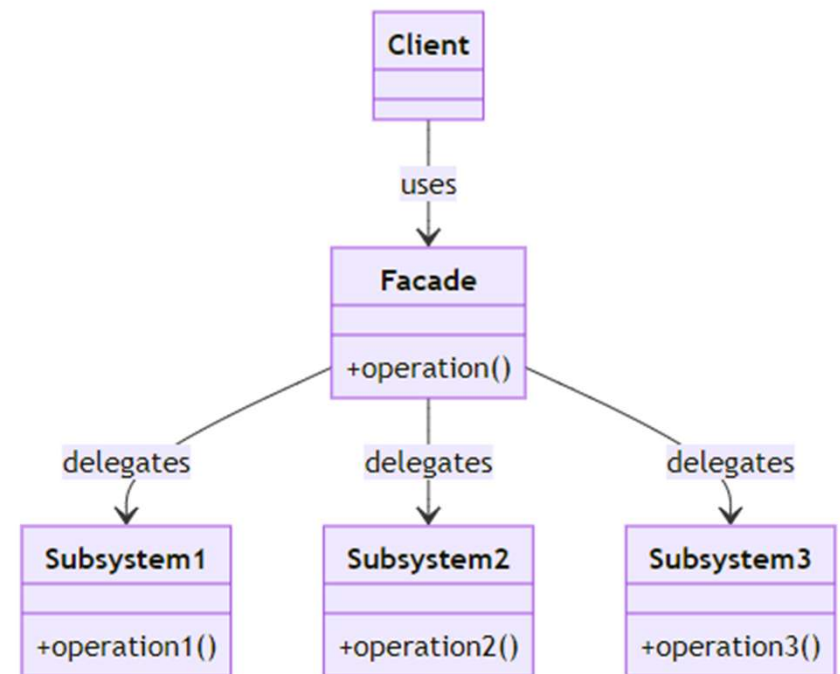
# Facade pattern

- Tên: facade
- Bài toán
- Giải pháp
  - Cung cấp một giao diện facade đơn giản để tương tác với một hệ thống phức tạp, giúp che giấu sự phức tạp và các chi tiết kỹ thuật không cần thiết khỏi client



# Facade pattern

- Cấu trúc:
  - Các hệ thống con
  - Lớp facade: biết rõ các hệ thống con
  - Client: sử dụng facade



# Facade pattern

## Cài đặt

### ■ Facade:

- nhiều class/sub system
- facade
- client

```
// Inventory subsystem
public class Inventory
{
    public bool CheckStock(string productId)
    { //check stock
        Console.WriteLine($"Checking stock for product {productId}");
        return true;
    }
}
```

# Facade pattern

## Cài đặt

Facade:

nhiều class/sub system  
facade  
client

```
public class Payment
{
    public bool ProcessPayment(string paymentDetails)
    {
        // Simulate payment processing
        Console.WriteLine($"Processing payment with details {paymentDetails}");
        return true;
    }
    //may using factory method pattern for diferent payment method
    public void PerformCODPayment(decimal amount)
    {
        Console.WriteLine($"Processing COD payment of {amount:C} ... DONE");
    }
    public void PerformCardPayment(decimal amount)
    {
        Console.WriteLine($"Processing Credit Card payment of {amount:C} ... DONE");
    }
}
```

# Facade pattern

## Cài đặt

Facade:

nhiều class/sub system  
facade  
client

```
public class ShopFacade
{
    private Inventory _inventory;
    private Payment _payment;
    private Shipping _shipping;
    public ShopFacade()
    {
        _inventory = new Inventory();
        _payment = new Payment();
        _shipping = new Shipping();
    }

    public void PlaceOrder(string productId, string paymentDetails) . . .
```

# Facade pattern

## Cài đặt

Facade:

nhiều class/sub system

facade

client

```
public class ShopFacade
{
    public void PlaceOrder(string productId, string paymentDetails)
    {
        if (_inventory.CheckStock(productId)){
            if (_payment.ProcessPayment(paymentDetails)){
                _shipping.ShipProduct(productId);
                Console.WriteLine("Order placed successfully.");
            }
            else {
                Console.WriteLine("Payment failed.");
            }
        }
        else {
            Console.WriteLine("Product out of stock.");
        }
    }
}
```

# Facade pattern

## Cài đặt

Facade:

nhiều class/sub system  
facade  
client

```
class Program
{
    static void Main(string[] args)
    {
        ShopFacade shop = new ShopFacade();
        shop.PlaceOrder("Product123", "PaymentDetailsXYZ");
    }
}
```

```
D:\Projects\source\repos4-hust\oop03\singleton02.cs
able reference type. [D:\Projects\source\repos4-hus
Checking stock for product Product123
Processing payment with details PaymentDetailsXYZ
Shipping product Product123
Order placed successfully.
PS D:\Projects\source\repos4-hust\oop03>
```

# Facade pattern

- Áp dụng
  - Khi có nhiều lớp => khó hiểu quy trình xử lý; khi có nhiều hệ thống con với các interface riêng rẽ - khó phối hợp
  - Khi phân lớp các hệ thống con
  - Khi muốn bao gói, che dấu sự phức tạp của hệ thống con đối với client

# Adapter pattern

## ■ VD

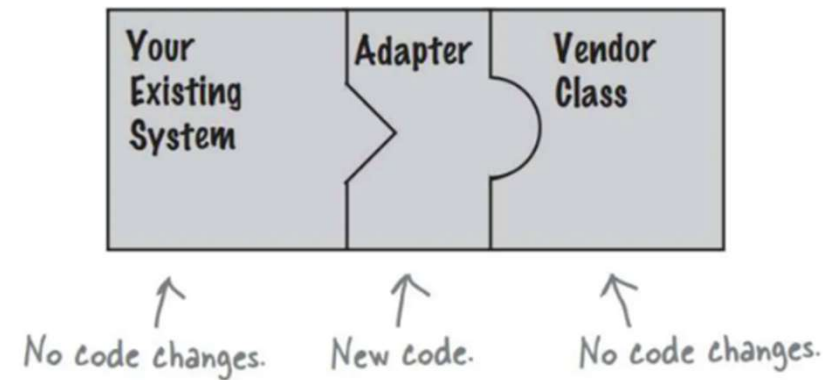
- Hệ thống (đang phát triển) cần tích hợp, tương tác hệ thống legacy cũ (hay gọi thư viện có sẵn). Tuy nhiên các interface không tương thích
- Có thể
  - Sửa đổi HT cũ ?
  - Sửa đổi HT mới ?
  - Sử dụng Adapter



Their interface doesn't match the one you've written your code against. This isn't going to work!

# Adapter pattern

- Giải pháp
  - Xây dựng adapter - vai trò kết nối, chuyển đổi interface => cho phép 2 hệ thống có thể tương tác (mà không cần thay đổi)

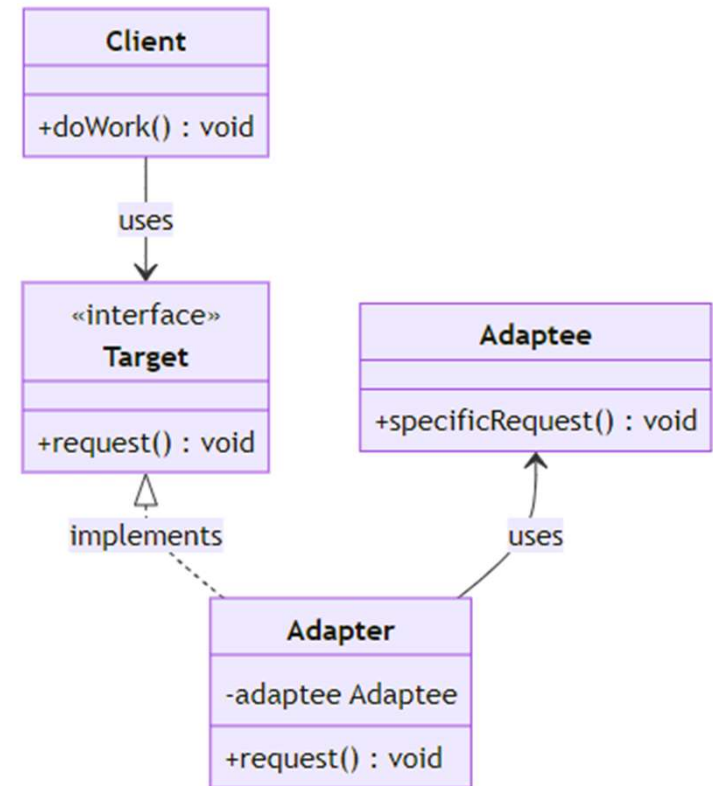


# Adapter pattern

- Tên: Adapter pattern
- Bài toán:
  - Cần tương tác với hệ thống/thư viện khác không tương thích
- Giải pháp:
  - Sử dụng (class) Adapter để chuyển đổi/diễn dịch sang interface tương thích HT cũ

# Adapter pattern

- Cấu trúc
  - Adaptee: HT/thư viện cũ cần tương tác
  - Client: HT mới cần tương tác với adaptee
  - Target: giao diện để client gọi tới
  - Adapter: lớp điều hợp, thực hiện chuyển đổi interface tương thích adaptee



# Adapter pattern

- Cài đặt

```
//adaptee - old interface/legacy
public class LegacyOrderSystem
{
    public void PlaceOrder(string order)
    {
        Console.WriteLine($"Order placed in OLD legacy system: {order}");
    }
}

//target interface - new interface
public interface IProcess
{
    void ProcessOrder(string orderDetails);
}
```

# Adapter pattern

- Cài đặt

```
//adapter
public class OrderAdapter : IProcess
{
    private readonly LegacyOrderSystem _legacyOrderSystem;
    public OrderAdapter(LegacyOrderSystem legacyOrderSystem)
    {
        _legacyOrderSystem = legacyOrderSystem;
    }
    public void ProcessOrder(string orderDetails)
    {
        // Translate the request to the adaptee's method (and back)
        _legacyOrderSystem.PlaceOrder(orderDetails);
    }
}
```

//client app

```
Console.WriteLine($"New Client needs to place an order");
LegacyOrderSystem legacyOrderSystem = new LegacyOrderSystem();
IProcess adapter = new OrderAdapter(legacyOrderSystem);
adapter.ProcessOrder("Order456");
```

```
D:\Projects\source\repos4-hust\oop03\facade01
never used [D:\Projects\source\repos4-hust\o
New Client needs to place an order
Order placed in OLD legacy system: Order456
PS D:\Projects\source\repos4-hust\oop03>
```

# Adapter pattern

- Lợi ích
  - Các đối tượng/HT có interface khác nhau có thể giao tiếp với nhau
- Sử dụng
  - Tương tác với HT cũ mà không cần thay đổi nó
  - Sử dụng lớp đã có nhưng interface không phù hợp
  - Chuyển đổi interface từ nhiều nguồn



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

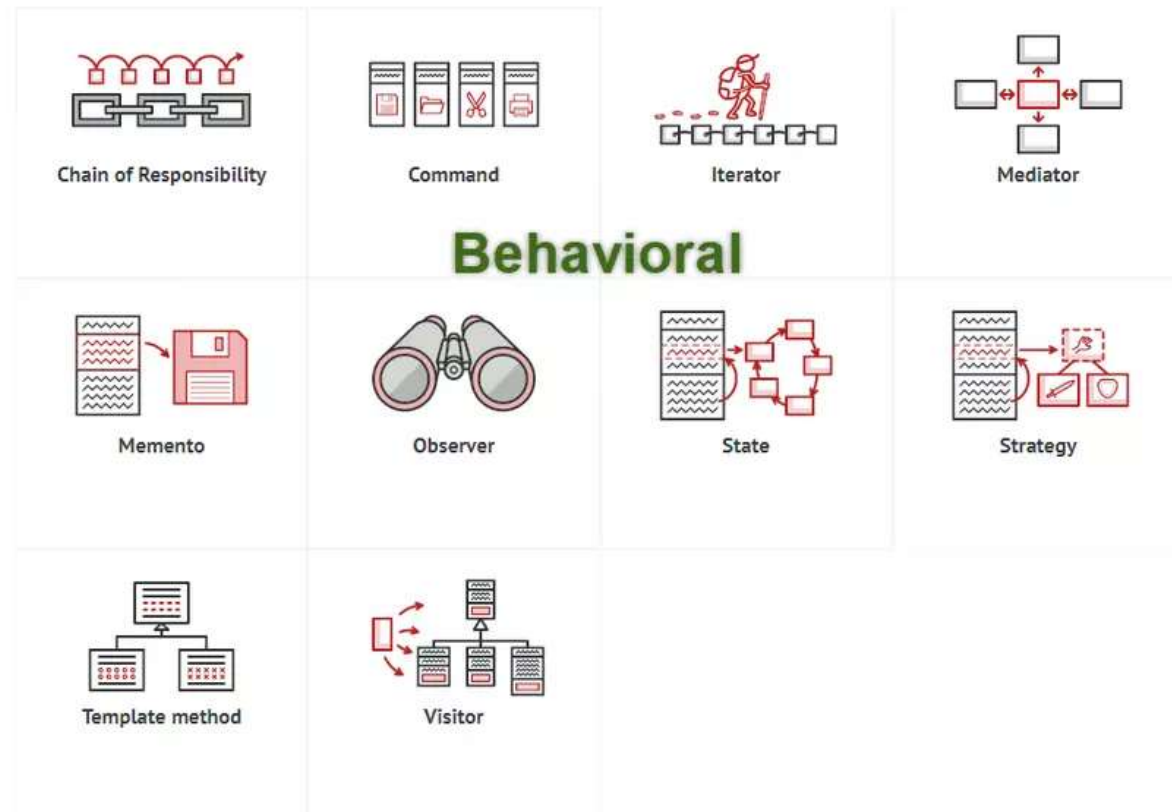
**KHOA TOÁN – TIN**

# Mẫu hành vi Behavioral Pattern



# Mẫu hành vi

- Giải quyết các vấn đề trong thực hiện hành vi các đối tượng giao tiếp với nhau



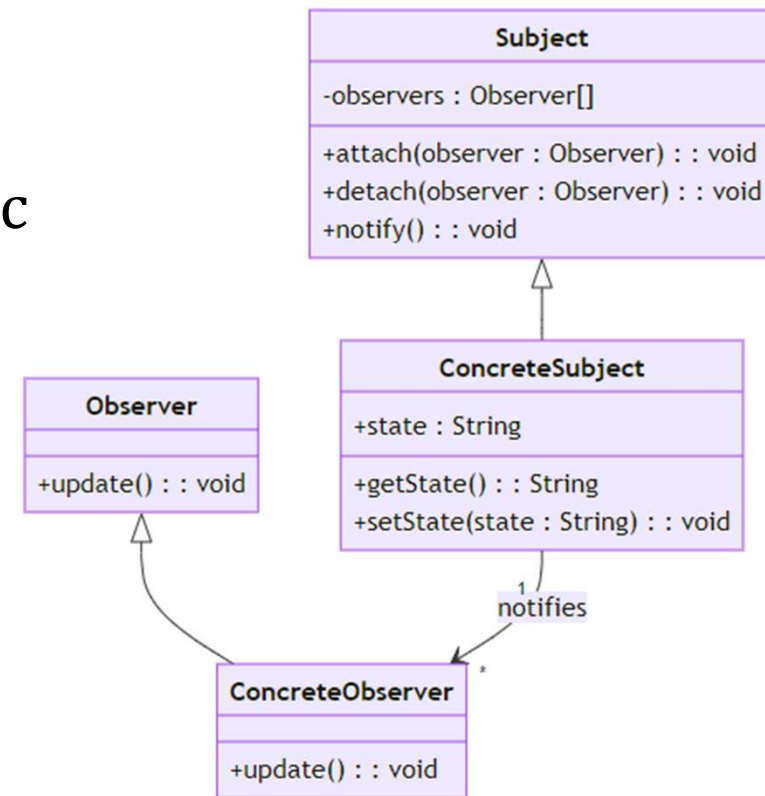
# Observer pattern

## ■ VD

- Khi xảy ra sự kiện/thay đổi trạng thái của một đối tượng => Các đối tượng khác cần được cập nhật (có thể thay đổi)
- Đối tượng account: khi có cập nhật => cần
  - Ghi log (bởi đối tượng logman)
  - Thông báo (bởi đối tượng mailer), ...
  - Thực hiện trong account sử dụng logman, mailer => khi có thay đổi !
- Giải pháp: observer pattern

# Mẫu Observer

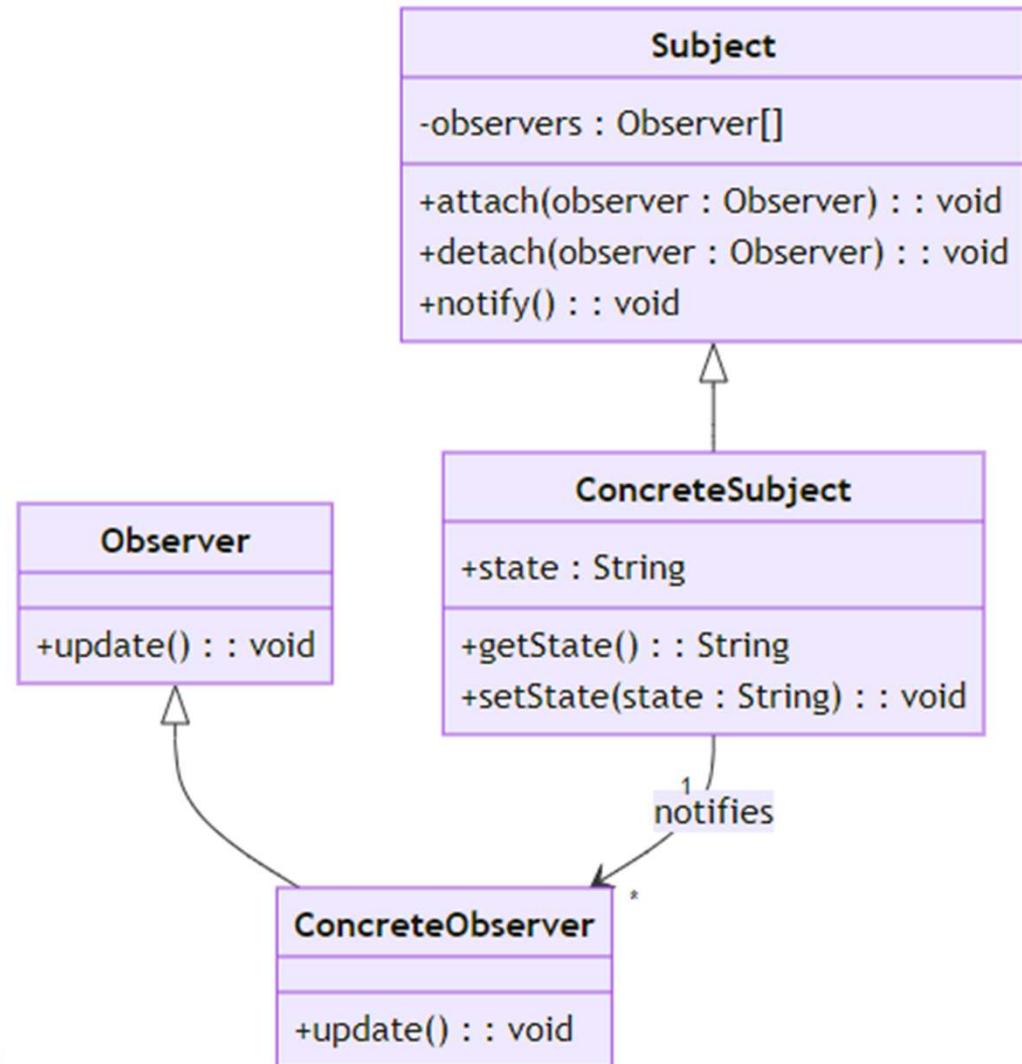
- Tên: Observer Pattern
- Vấn đề: Nhiều đối tượng phụ thuộc vào một đối tượng khác và khi đối tượng khác này thay đổi, các đối tượng phụ thuộc nó cần được thông báo (cập nhật)
- Giải pháp
  - Observer: cập nhật tới đối tượng quan tâm
  - Subject: đối tượng cần theo dõi, quản lý danh sách các observer



# Observer pattern

## ■ Các thành phần

- Subject: đối tượng chủ thể cần  
Nó duy trì danh sách các Observer  
thông báo cho chúng khi trạng  
đổi.
- Observer: đối tượng quan sát  
đăng ký theo dõi Subject và cập  
nhận được thông báo từ Subject
- ConcreteSubject và ConcreteObserver  
các cài đặt cụ thể.



# Observer pattern

- Cài đặt

```
public interface IObservable  
{  
    void Update();  
}
```

```
public interface ISubject  
{  
    void Attach(IObservable observable);  
    void Detach(IObservable observable);  
    void Notify();  
}
```

# Observer pattern

```
public class UserAccount : ISubject
{
    private List<IObserver> observers = new List<IObserver>();
    private string state;
    public string State{
        get { return state; }
        set {
            state = value;
            Notify();
        }
    }
    public void Attach(IObserver observer){
        observers.Add(observer);
    }
    public void Detach(IObserver observer){
        observers.Remove(observer);
    }
    public void Notify(){
        foreach (var observer in observers){
            observer.Update();
        }
    }
}
```



# Observer pattern

```
public class Mailer : IObservable
{
    private UserAccount userAccount;
    public Mailer(UserAccount userAccount)
    {
        this.userAccount = userAccount;
        this.userAccount.Attach(this);
    }
    public void Update()
    {
        Console.WriteLine($"Mailer: Dear Sir, account state changed to: {userAccount.State}");
    }
}
```

# Observer pattern

```
public class Logger : IObserver
{
    private UserAccount userAccount;
    public Logger(UserAccount userAccount)
    {
        this.userAccount = userAccount;
        this.userAccount.Attach(this);
    }
    public void Update()
    {
        Console.WriteLine($"Logger: User account state changed to: {userAccount.State}");
    }
}
```

# Observer pattern

```
Console.WriteLine($"***** Observer pattern *****");  
UserAccount userAccount = new UserAccount();
```

```
Mailer mailer = new Mailer(userAccount);  
Logger logger = new Logger(userAccount);  
Console.WriteLine($"--- Setting user account state to Active");  
userAccount.State = "Active";  
Console.WriteLine($"--- Setting user account state to Suspended");  
userAccount.State = "Suspended";
```

```
***** Observer pattern *****  
--- Setting user account state to Active  
Mailer: Dear sir, account state changed to: Active  
Logger: User account state changed to: Active  
--- Setting user account state to Suspended  
Mailer: Dear sir, account state changed to: Suspended  
Logger: User account state changed to: Suspended  
PS D:\Projects\source\repos4-hust\oop03> █
```

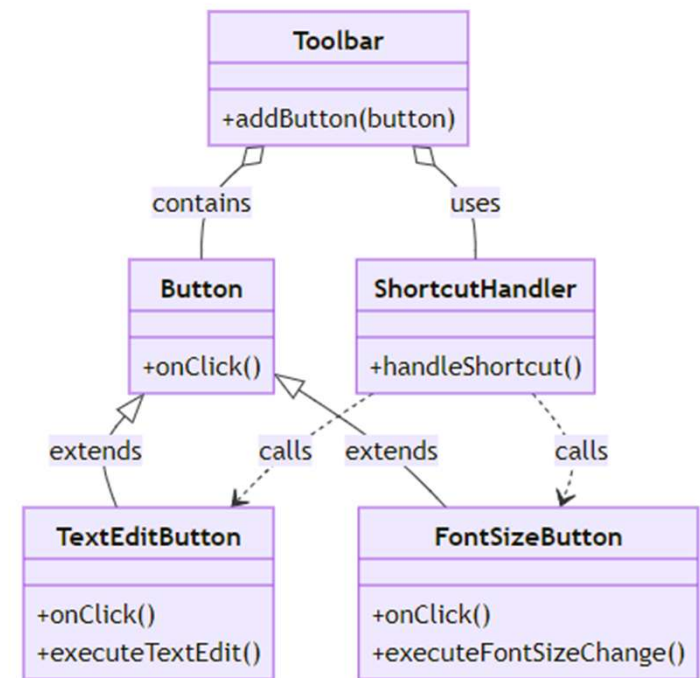
# Command pattern

## ■ Bài toán

- Cần gửi yc cho đối tượng mà kh biết về hành động và đối tượng nhận yêu cầu:
- Trình soạn thảo với toolbar - có các nút lệnh với chức năng khác nhau
  - Tạo lớp cơ sở button, các lệnh khác nhau => lớp thừa kế
  - Cần thêm phím tắt => trùng lặp, phức tạp

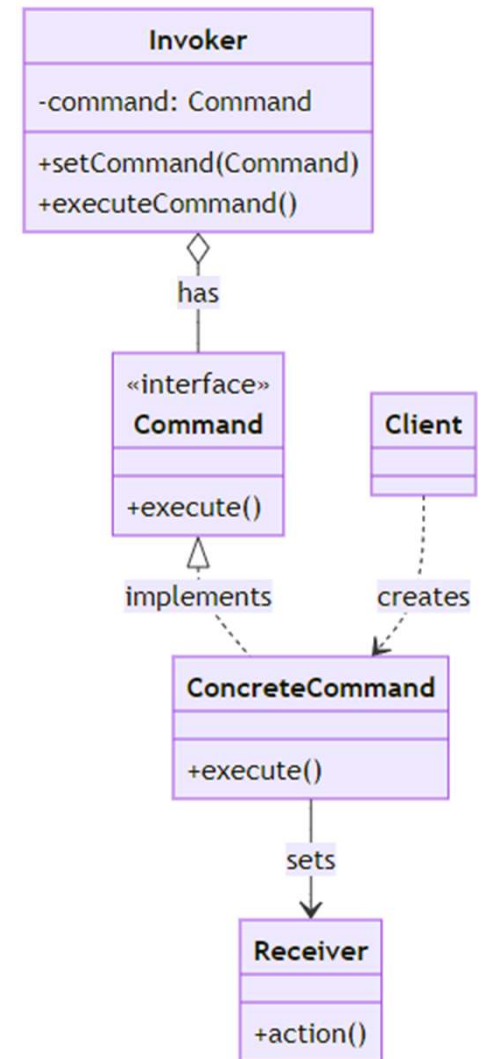
## ■ Giải pháp

- Đóng gói yêu cầu thành đối tượng command chứa thông tin cần thiết và tên method cần thực hiện



# Command pattern

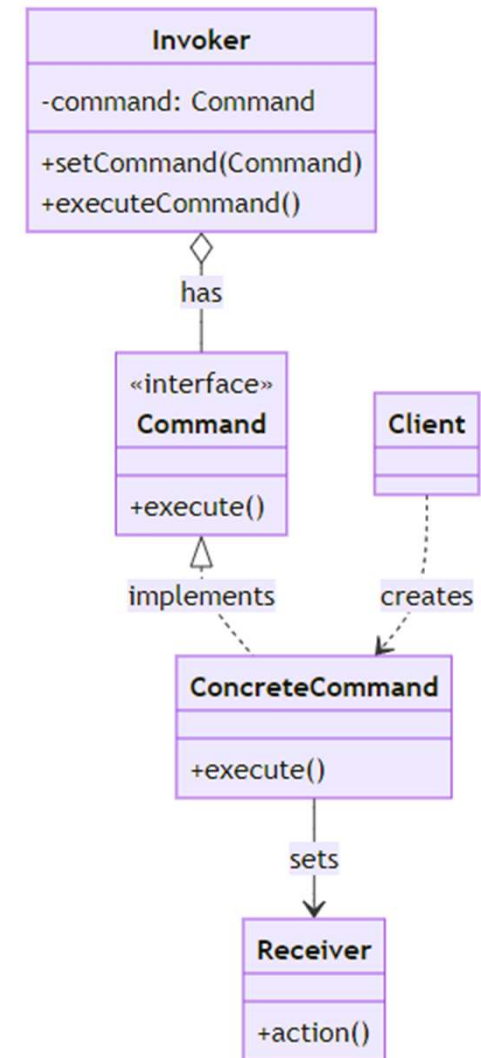
- Tên: Command pattern
- Vấn đề
  - Liên hệ phức tạp giữa bên yêu cầu và đối tượng thực hiện
- Giải pháp
  - Tách biệt bên yc và đối tượng thực hiện yc
  - Đóng gói thông tin yc trong đối tượng command



# Command pattern

## ■ Cấu phần

- Command interface: Định nghĩa method execute(), ...
- ConcreteCommand:
  - lớp cụ thể thực thi command
  - Method execute() => gọi hành động của đối tượng thực thi Receiver
- Invoker:
  - lưu tham chiếu của command cụ thể
  - Gọi method execute() trên đối tượng command
- Receiver:
  - Đối tượng thực thi, biết cách thực thi. Được gọi qua method action()
- Client:
  - Đóng gói yc thành ConcreteCommand cụ thể phù hợp
  - Thiết lập receiver của command và gọi Invoke



# Command pattern

## Cài đặt

```
public interface ICommand
{
    void Execute();
    void Undo();
}
```

```
// ConcreteCommand: VD lệnh mở tài khoản - trên BankAccount
public class OpenAccountCommand : ICommand
{
    private BankAccount _bankAccount;
    //set receiver
    public OpenAccountCommand(BankAccount bankAccount)
    {
        _bankAccount = bankAccount;
    }
    //execute method: yêu cầu receiver thực hiện hành động
    public void Execute()
    {
        _bankAccount.OpenAccount();
    }
    public void Undo()
    {
        _bankAccount.CloseAccount();
    }
}
```

# Command pattern

## Cài đặt

```
// Receiver Class
public class BankAccount
{
    public string AccountNumber { get; private set; }
    public decimal Balance { get; private set; }
    public BankAccount(string accountNumber){
        AccountNumber = accountNumber; Balance = 0;
    }
    //action: hành động thực hiện của receiver
    public void OpenAccount(){
        Console.WriteLine($"Open account: {AccountNumber}!");
    }
    //another action: hành động đóng tài khoản của receiver
    public void CloseAccount(){ ... }
    //another action: hành động nộp tiền
    public void Deposit(decimal amount){
        Balance += amount;
        Console.WriteLine($"Deposit {amount} to account {AccountNumber}. New Balance: {Balance}");
    }
}
```

# Command pattern

## Cài đặt

//Một ConcreteCommand khác: lệnh đóng tài khoản

```
public class CloseAccountCommand : ICommand
{ ... }
```

//Một ConcreteCommand khác: lệnh nộp tiền

```
public class DepositCommand : ICommand
{
    private BankAccount _bankAccount;
    private decimal _amount;
    public DepositCommand(BankAccount bankAccount, decimal amount){
        _bankAccount = bankAccount;
        _amount = amount;
    }
    public void Execute(){
        _bankAccount.Deposit(_amount);
    } ...
}
```

//Một ConcreteCommand khác: lệnh rút tiền

```
public class WithdrawCommand : ICommand
{ ... }
```

# Command pattern

## Cài đặt

```
// Invoker Class
public class BankManager
{
    //có thể dùng stack lưu lịch sử các lệnh đã thực thi => cho phép undo
    private readonly Stack<ICommand> _commandHistory = new Stack<ICommand>();
    //Invoker gọi method Execute của Command
    public void ExecuteCommand(ICommand command) {
        Console.WriteLine("### Execute command: ");
        command.Execute();
        _commandHistory.Push(command);
    }
    //Nếu cho undo: command interface cũng có method Undo
    public void UndoLastCommand(){
        Console.WriteLine("### Undo last command: ");
        ICommand lastCommand = _commandHistory.Pop();
        lastCommand.Undo();
    }
}
```

# Command pattern

## Cài đặt

```
/******
```

```
Command pattern: Client app
```

```
*****/
```

```
Console.WriteLine($"***** COMMAND Pattern *****");
```

```
BankAccount account = new BankAccount("123456");
```

```
BankManager manager = new BankManager();
```

```
ICommand openAccount = new OpenAccountCommand(account);
```

```
ICommand deposit = new DepositCommand(account, 100);
```

```
ICommand withdraw = new WithdrawCommand(account, 50);
```

```
ICommand closeAccount = new CloseAccountCommand(account);
```

```
manager.ExecuteCommand(openAccount);
```

```
manager.ExecuteCommand(deposit);
```

```
manager.ExecuteCommand(withdraw);
```

```
manager.ExecuteCommand(closeAccount);
```

```
***** COMMAND Pattern *****
### Execute command:
Open account: 123456!
### Execute command:
Deposit 100 to account 123456. New Balance: 100
### Execute command:
Withdraw 50 from account 123456. New balance: 50
### Execute command:
Close account: 123456!
### Undo last command:
Open account: 123456!
### Undo last command:
Deposit 50 to account 123456. New Balance: 100
PS D:\Projects\source\repos4-hust\oop03> █
```



# Bài tập

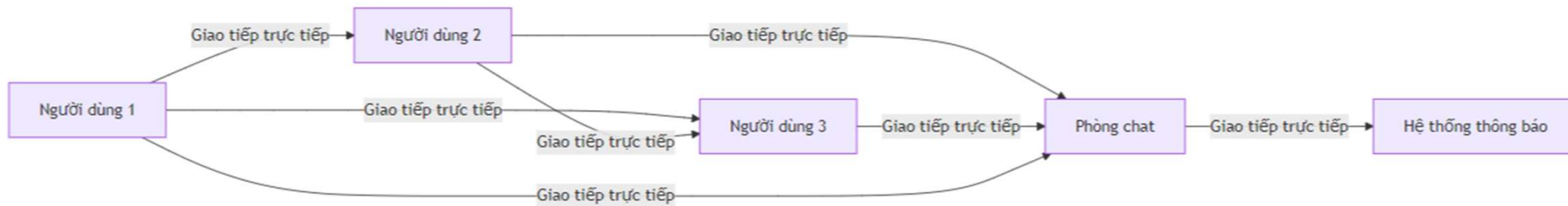
- Sinh viên nhập học thành công => cần báo cho đối tượng kế toán và đối tượng thư viện
- Cài đặt observer pattern



# Mediator pattern

## ■ Bài toán

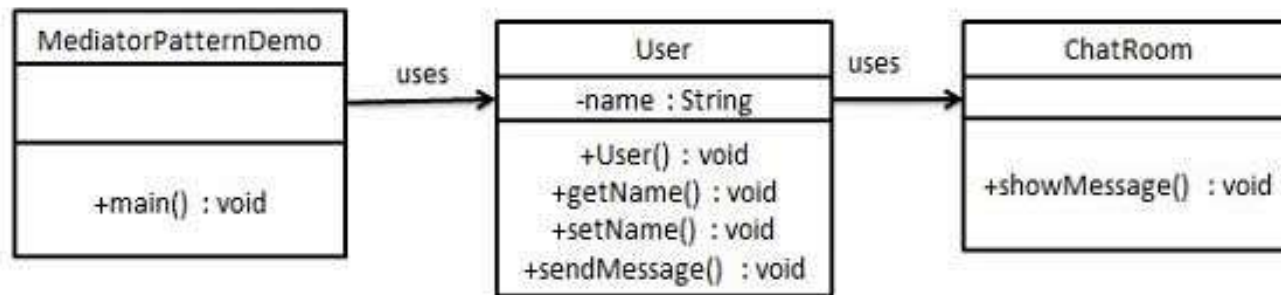
- Ứng dụng: nhiều đối tượng tương tác với nhau
- Nếu các đối tượng luôn gọi trực tiếp => có thể mã sẽ khó quản lý



- Giải pháp: mediator

# Mediator Pattern

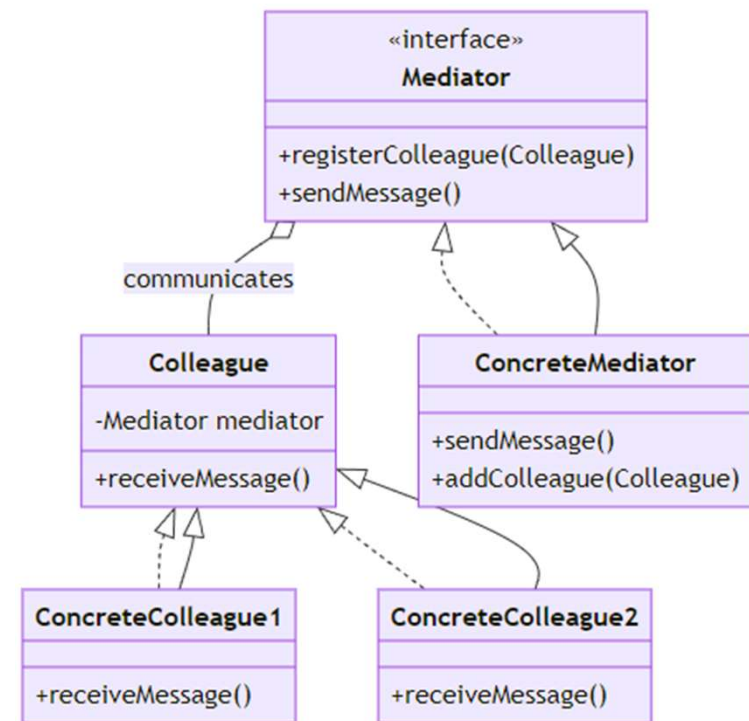
- Tên: Mediator Pattern
- Vấn đề cần giải quyết:
  - giao tiếp phức tạp giữa nhiều đối tượng hay lớp
- Giải pháp:
  - tạo lớp hỗ trợ giao tiếp giữa các lớp/đối tượng khác - mediator



# Mediator Pattern

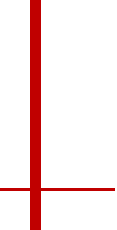
## ■ Cấu phần

- Colleague: abstract class, có ref tới mediator (giao tiếp qua mediator thay vì giao tiếp trực tiếp với các colleague khác)
- ConcreteColleague: cài đặt method của Colleague. Giao tiếp thông qua Mediator khi cần giao tiếp với Colleague khác
- Mediator: interface để giao tiếp với các Colleague
- ConcreteMediator : cài đặt các phương thức của Mediator, biết và quản lý các Colleague object



```
public interface IChatMediator{
    void SendMessage(string message, User user);
    void AddUser(User user);
}

// Concrete Mediator
public class ChatMediator : IChatMediator
{
    private List<User> users;
    public ChatMediator(){
        users = new List<User>();
    }
    public void AddUser(User user){
        users.Add(user);
    }
    public void SendMessage(string message, User user)
    {
        foreach (var u in users){
            if (u != user){
                u.Receive(message);
            }
        }
    }
}
```

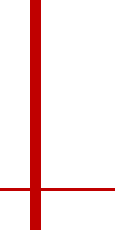


```
// Abstract Colleague
public abstract class User
{
    protected IChatMediator mediator;
    protected string name;

    public User(IChatMediator mediator, string name)
    {
        this.mediator = mediator;
        this.name = name;
    }

    public abstract void Send(string message);
    public abstract void Receive(string message);
}
```



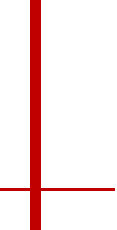


```
public class ChatUser : User
{
    public ChatUser(IChatMediator mediator, string name) :
base(mediator, name) { }

    public override void Send(string message)
    {
        Console.WriteLine(this.name + " sends: " + message);
        mediator.SendMessage(message, this);
    }

    public override void Receive(string message)
    {
        Console.WriteLine(this.name + " receives: " + message);
    }
}
```





```
//Client
```

```
static void Main(string[] args)
{
    IChatMediator chatMediator = new ChatMediator();

    User user1 = new ChatUser(chatMediator, "User1");
    User user2 = new ChatUser(chatMediator, "User2");
    User user3 = new ChatUser(chatMediator, "User3");
    User user4 = new ChatUser(chatMediator, "User4");

    chatMediator.AddUser(user1);
    chatMediator.AddUser(user2);
    chatMediator.AddUser(user3);
    chatMediator.AddUser(user4);

    user1.Send("Hello, everyone!");
    user2.Send("Hi, User1!");
}
```





- VD java

```
import java.util.Date;

public class ChatRoom {
    public static void showMessage(User user, String message){
        System.out.println(new Date().toString() + " [" + user.getName()
+ "]" : " + message);
    }
}
```

```
public class User {
    private String name;
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }

    public User(String name){
        this.name = name;
    }

    public void sendMessage(String message){
        ChatRoom.showMessage(this,message);
    }
}
```



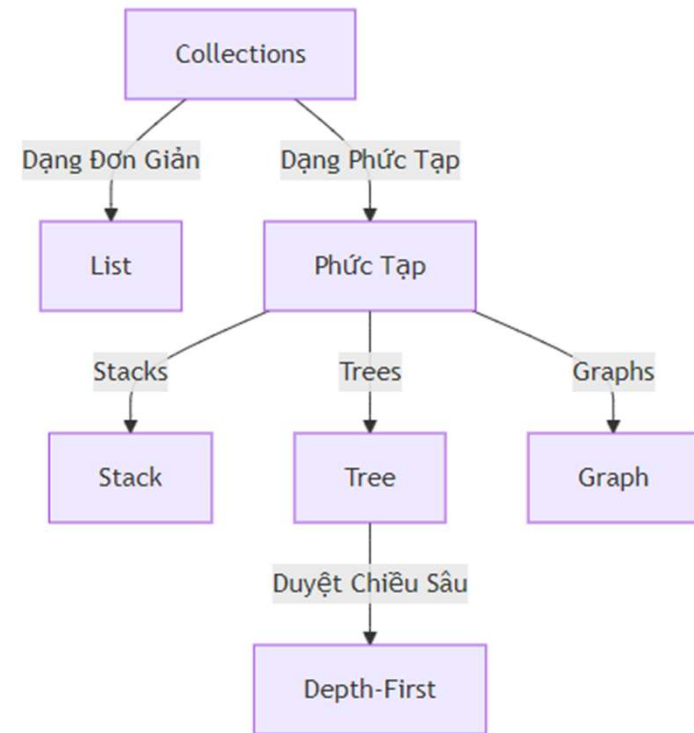
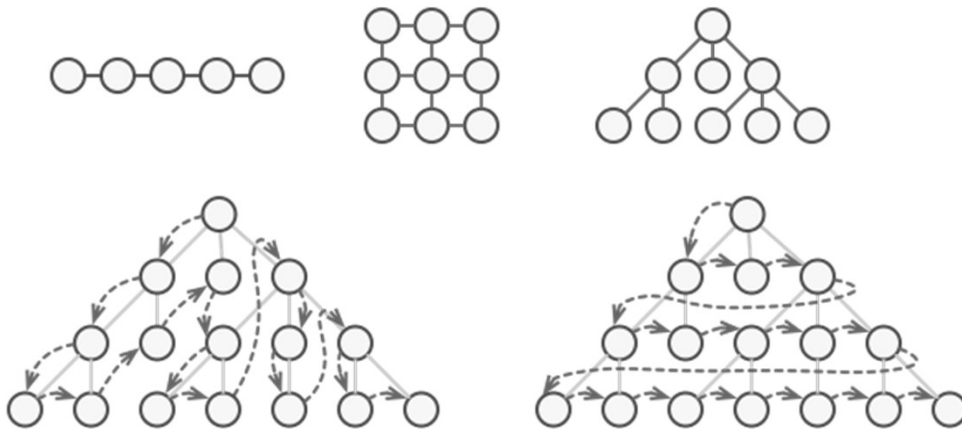
- VD java

```
public class MediatorPatternDemo {  
    public static void main(String[] args) {  
        User robert = new User("Robert");  
        User john = new User("John");  
  
        robert.sendMessage("Hi! John!");  
        john.sendMessage("Hello! Robert!");  
    }  
}
```

```
Thu Jan 31 16:05:46 IST 2023 [Robert] : Hi! John!  
Thu Jan 31 16:05:46 IST 2023 [John] : Hello! Robert!
```

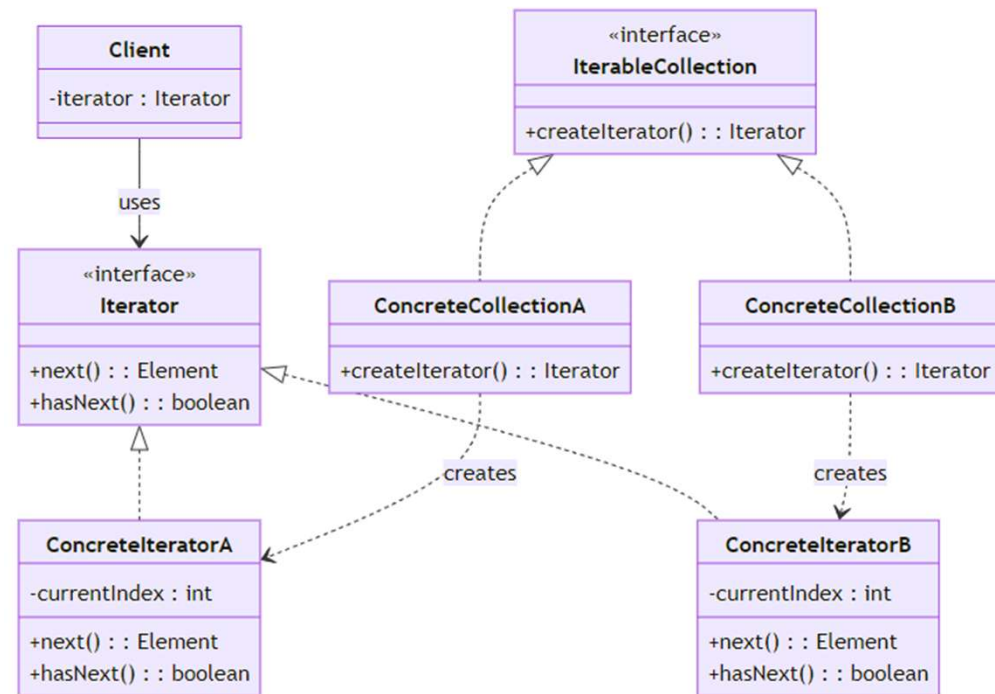
# Iterator pattern

- Nhu cầu duyệt các phần tử của tập hợp (collection) mà không cần quan tâm cấu trúc cụ thể



# Iterator pattern

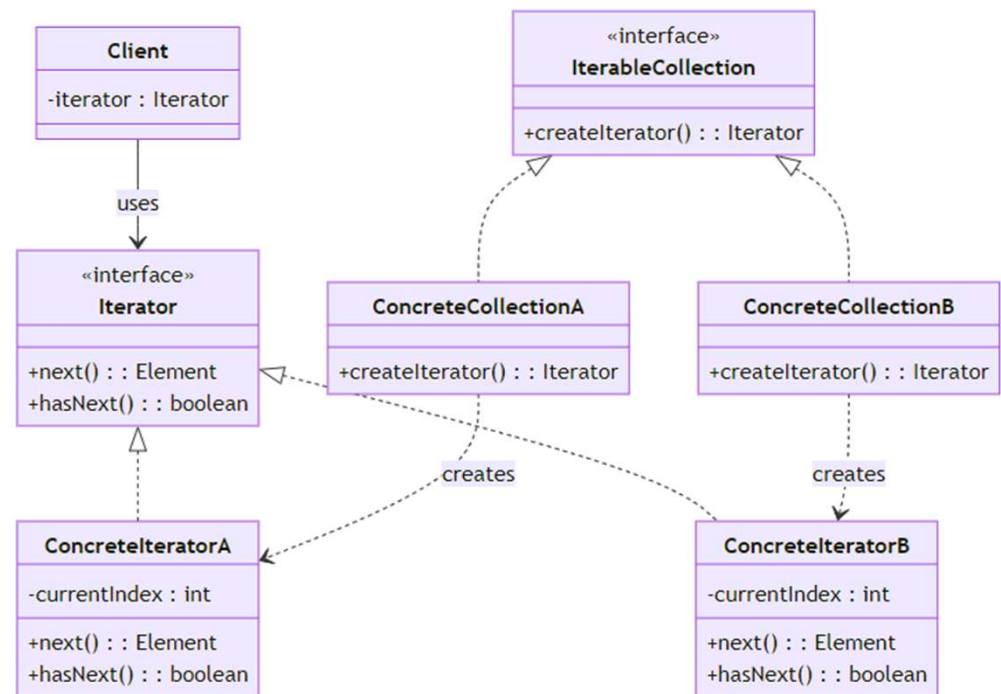
- Tên:
  - Iterator Pattern
- Vấn đề cần giải quyết:
  - Truy cập tới các phần tử của đối tượng collection
- Giải pháp:
  - Tạo lớp Iterator, tách biệt hành vi duyệt ra khỏi cấu trúc phần tử



# Iterator pattern

## ■ Cấu phần

- Iterator: interface các method duyệt
- Concrete Iterator: cài đặt iterator, lưu index duyệt
- Iterable Collection: interface các method tạo iterator
- Concrete Collections: cài đặt Iterable Collection, trả về Concrete Iterator phù hợp
- Client: trình khách





# Iterator pattern

- Cài đặt



```
public interface Iterator {  
    public boolean hasNext();  
    public Object next();  
}
```

```
public interface Container {  
    public Iterator getIterator();  
}
```

```

public class NameRepository implements Container {
    public String names[] = {"Robert" , "John" , "Julie" , "Lora"};
    @Override
    public Iterator getIterator() {
        return new NameIterator();
    }
    private class NameIterator implements Iterator {
        int index;
        @Override
        public boolean hasNext() {
            if(index < names.length){
                return true;
            }
            return false;
        }
        @Override
        public Object next() {
            if(this.hasNext()){
                return names[index++];
            }
            return null;
        }
    }
}

```

```
public class IteratorPatternDemo {  
  
    public static void main(String[] args) {  
        NameRepository namesRepository = new NameRepository();  
  
        for(Iterator iter = namesRepository.getIterator();  
            iter.hasNext();){  
            String name = (String)iter.next();  
            System.out.println("Name : " + name);  
        }  
    }  
}
```

```
Name : Robert  
Name : John  
Name : Julie  
Name : Lora
```

# Mẫu MVVM

- Tên: Model-View-ViewModel Pattern
- Vấn đề cần giải quyết: phân tách các thành phần của hệ thống
- Giải pháp: tạo lớp 3 lớp, Model, View và ViewModel

